□□□□□□

Lecture #1: Introduction to AI

Offered: 2025-02-25

Jia-Li Yin

Department of Computer Science and Big Data

Fuzhou University, Fuzhou, China

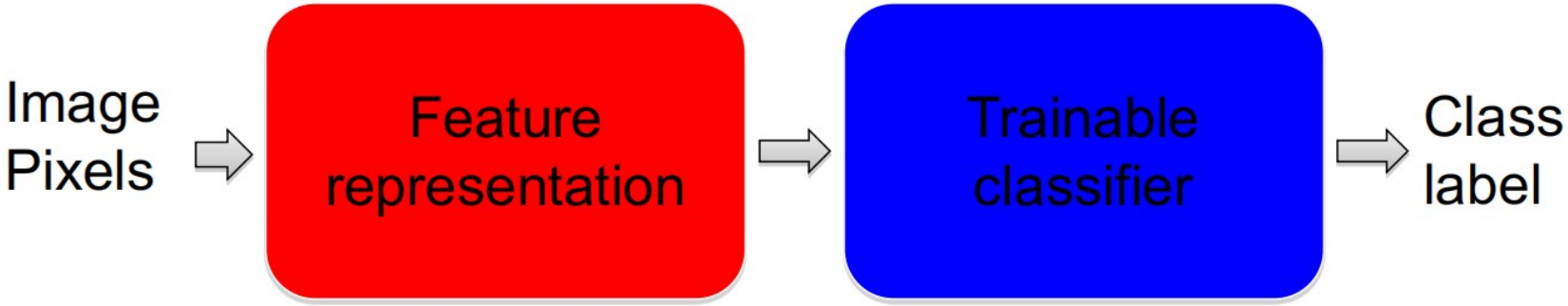
jlyin@fzu.edu.cn

Outline

- Perceptrons
- Multi-layer neural networks
- • Training method
- Best practices for training classifiers
- After that: convolutional neural networks

Recall: “Shallow” recognition pipeline

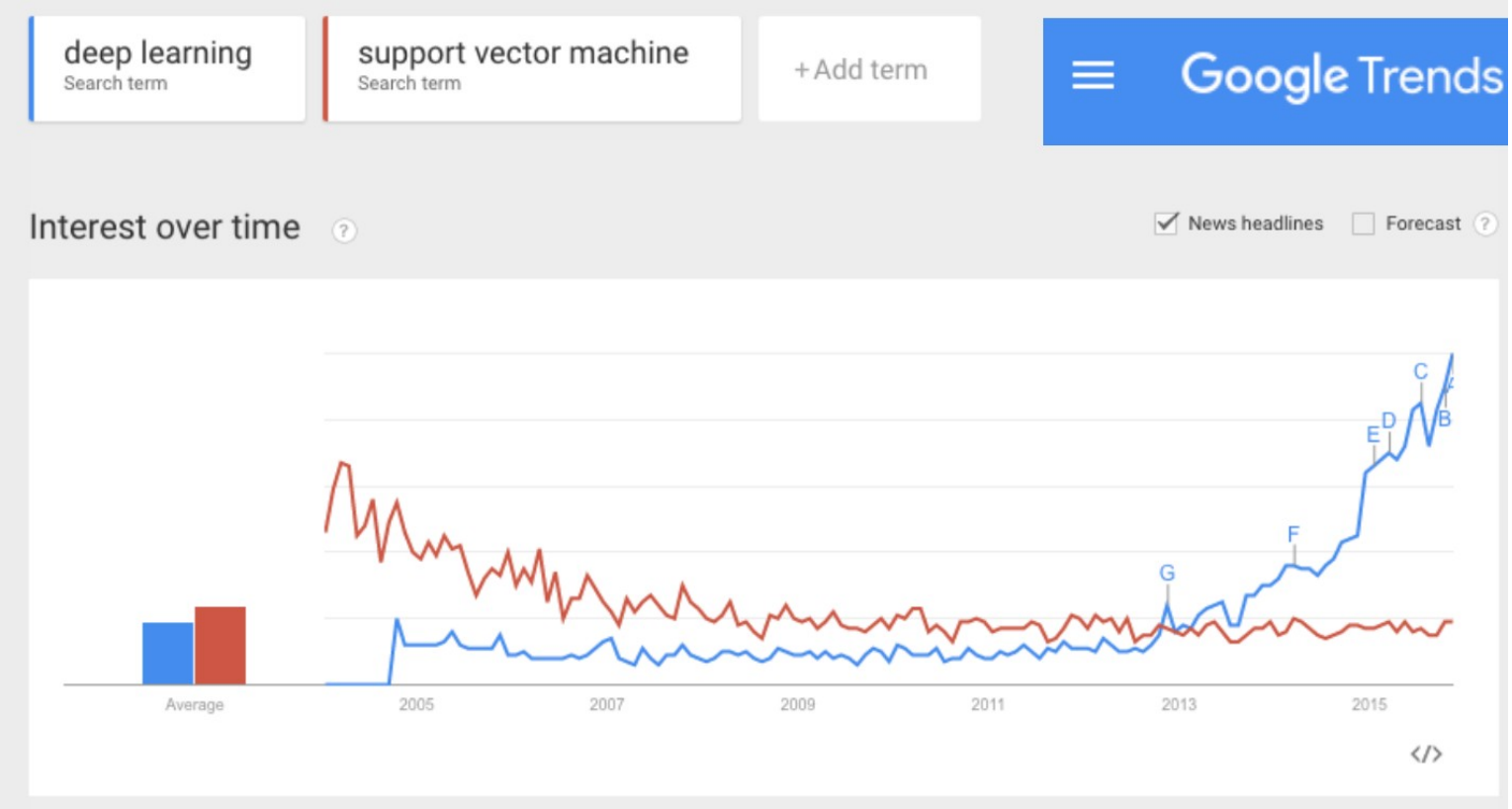
- Hand-crafted feature representation
- Off-the-shelf trainable classifier



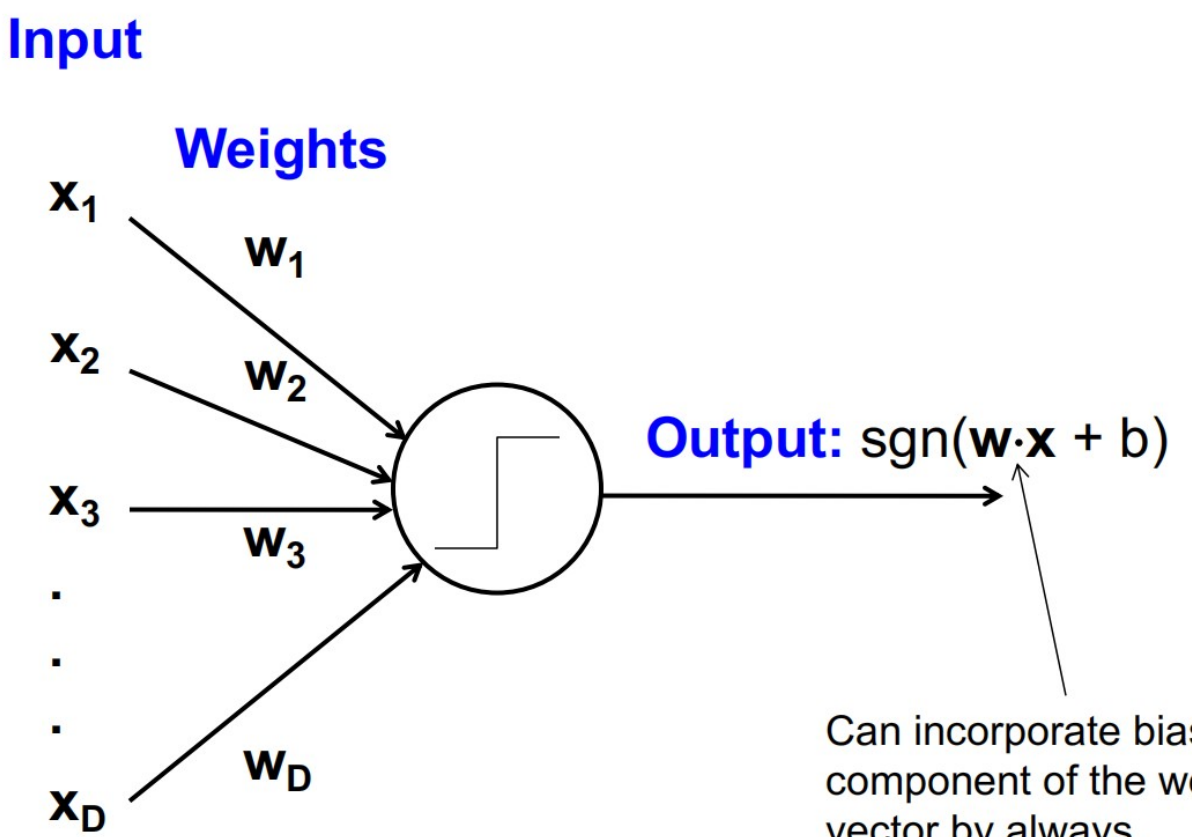
“Deep” recognition pipeline

- • Learn a feature hierarchy from pixels to classifier
- • Each layer extracts features from the output of previous layer
- • Train all layers jointly

Neural networks vs. SVMs
(a.k.a. “deep” vs. “shallow” learning)

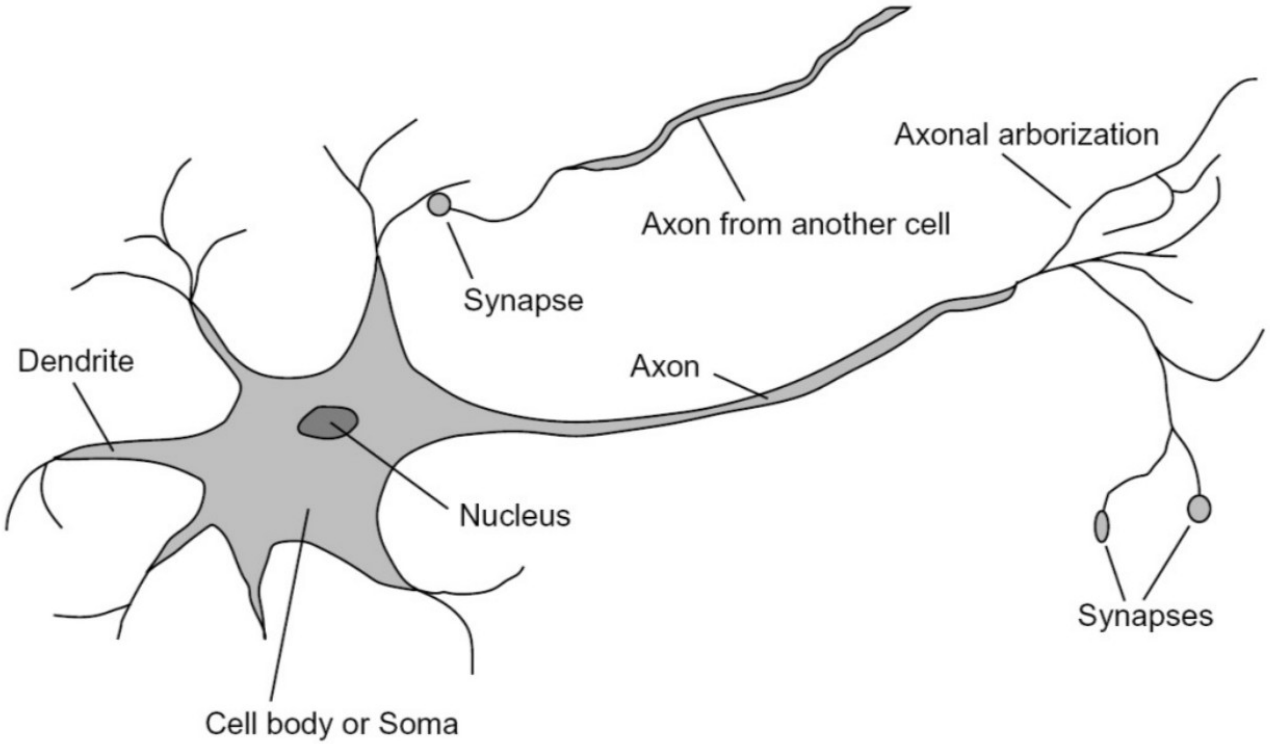


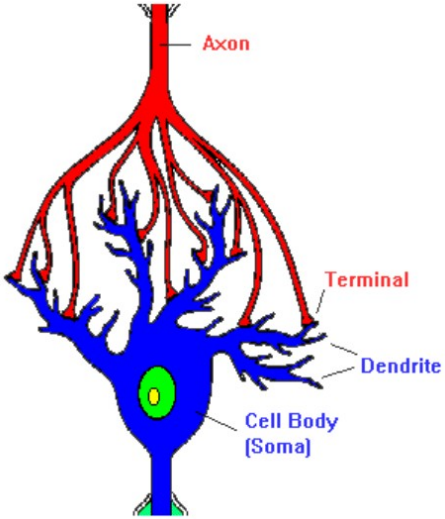
Linear classifiers revisited: Perceptron

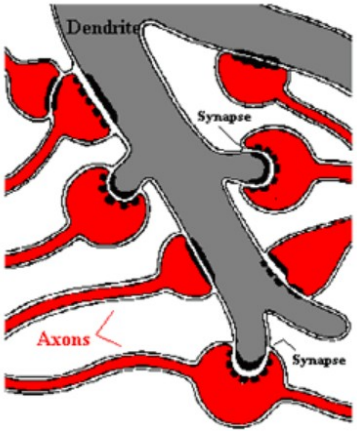


Can incorporate bias as component of the weight vector by always including a feature with value set to 1

Loose inspiration: Human neurons







Loose inspiration: Human neurons

1958 New York Times…

NEW NAVY DEVICE LEARNS BY DOING

Psychologist Shows Embryo of Computer Designed to Read and Grow Wiser

WASHINGTON, July 7 (UPI) —The Navy revealed the embryo of an electronic computer today that it expects will be able to walk, talk, see, write, reproduce itself and be conscious of its existence.

The embryo—the Weather Bureau's $2,000,000 "704" computer—learned to differentiate between right and left after fifty attempts in the Navy's demonstration for newsmen.

The service said it would use this principle to build the first of its Perceptron thinking machines that will be able to read and write. It is expected to be finished in about a year at a cost of $100,000.

Dr. Frank Rosenblatt, designer of the Perceptron, conducted the demonstration. He said the machine would be the first device to think as the human brain. As do human beings, Perceptron will make mistakes at first, but will grow wiser as it gains experience, he said.

Dr. Rosenblatt, a research psychologist at the Cornell Aeronautical Laboratory, Buffalo, said Perceptrons might be fired to the planets as mechanical space explorers.

Without Human Controls

The Navy said the perceptron would be the first non-living mechanism "capable of receiving, recognizing and identifying its surroundings without any human training or control."

The "brain" is designed to remember images and information it has perceived itself. Ordinary computers remember only what is fed into them on punch cards or magnetic tape.

Later Perceptrons will be able to recognize people and call out their names and instantly translate speech in one language to speech or writing in another language, it was predicted.

Mr. Rosenblatt said in principle it would be possible to build brains that could reproduce themselves on an assembly line and which would be conscious of their existence.

In today's demonstration, the "704" was fed two cards, one with squares marked on the left side and the other with squares on the right side.

Learns by Doing

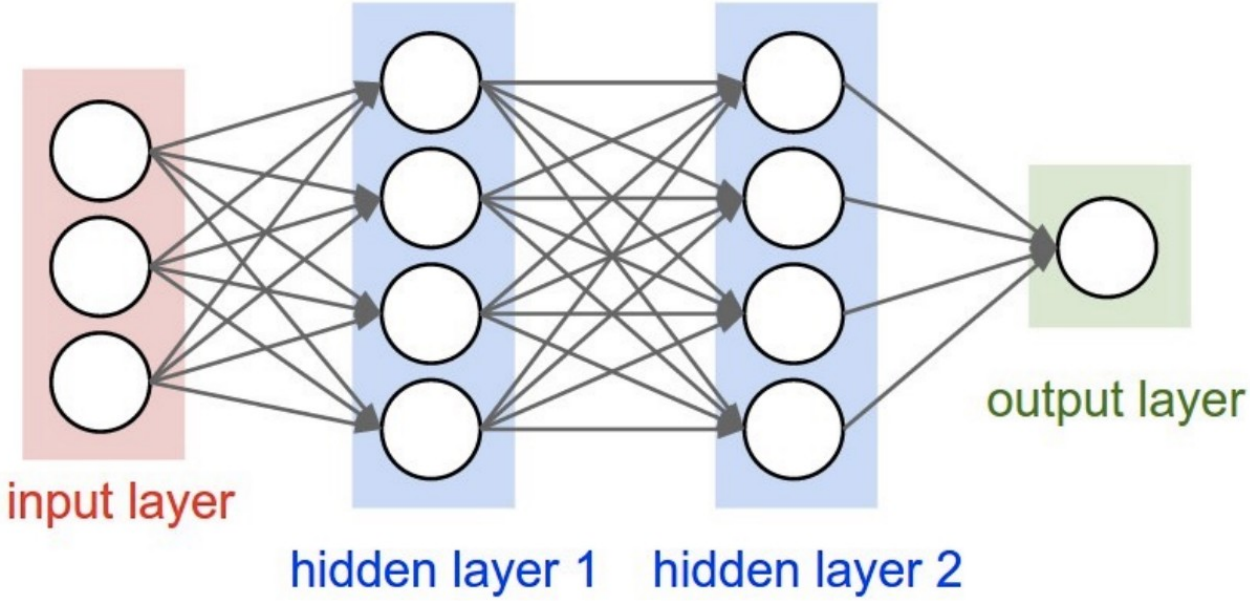
In the first fifty trials, the machine made no distinction between them. It then started registering a "Q" for the left squares and "O" for the right squares.

Dr. Rosenblatt said he could explain why the machine learned only in highly technical terms. But he said the computer had undergone a "self-induced change in the wiring diagram."

The first Perceptron will have about 1,000 electronic "association cells" receiving electrical impulses from an eye-like scanning device with 400 photo-cells. The human brain has 10,000,000,000 responsive cells, including 100,000,000 connections with the eyes.

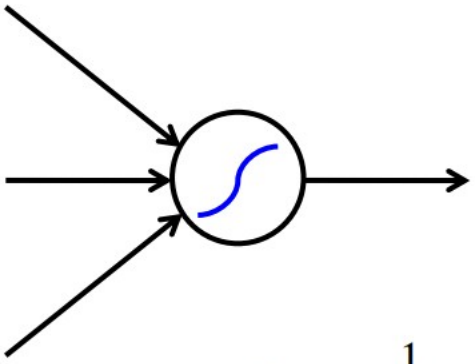
Multi-layer perceptrons

- To make nonlinear classifiers out of perceptrons, build a multi-layer neural network!
- This requires each perceptron to have a nonlinearity

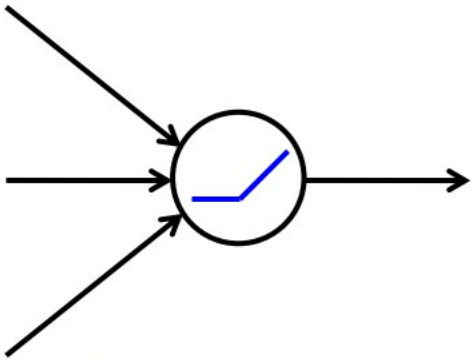


Multi-layer perceptrons

- To make nonlinear classifiers out of perceptrons, build a multi-layer neural network!
- This requires each perceptron to have a nonlinearity
- To be trainable, the nonlinearity should be differentiable



Sigmoid: $g(t) = \frac{1}{1+e^{-t}}$



Rectified linear unit (ReLU): $g(t) = \max(0,t)$

Training of multi-layer networks

- Find network weights to minimize the prediction loss between true and estimated labels of training examples:

$$E(\mathbf{w}) = \sum_i l(\mathbf{x}_i, y_i; \mathbf{w})$$

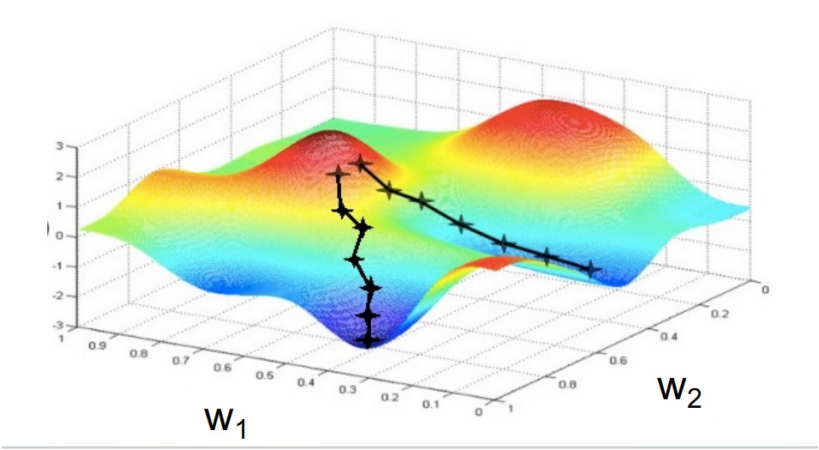
- Possible losses (for binary problems):
  - Quadratic loss: $l(\mathbf{x}_i, y_i; \mathbf{w}) = (f_{\mathbf{w}}(\mathbf{x}_i) - y_i)^2$
  - Log likelihood loss: $l(\mathbf{x}_i, y_i; \mathbf{w}) = -\log P_{\mathbf{w}}(y_i \mid \mathbf{x}_i)$
  - Hinge loss: $l(\mathbf{x}_i, y_i; \mathbf{w}) = \max(0, 1 - y_i f_{\mathbf{w}}(\mathbf{x}_i))$

Training of multi-layer networks

- Find network weights to minimize the prediction loss between true and estimated labels of training examples:

$$E(\mathbf{w}) = \sum_i l(\mathbf{x}_i, y_i; \mathbf{w})$$

- Update weights by gradient descent: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial E}{\partial \mathbf{w}}$



Training of multi-layer networks

- Find network weights to minimize the prediction loss between true and estimated labels of training examples:

$$E(\mathbf{w}) = \sum_i l(\mathbf{x}_i, y_i; \mathbf{w})$$

- Update weights by gradient descent:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial E}{\partial \mathbf{w}}$$

- **Back-propagation**: gradients are computed in the direction from output to input layers and combined using chain rule

- **Stochastic gradient descent**: compute the weight update w.r.t. one training example (or a small batch of examples) at a time, cycle through training examples in random order in multiple epochs

Back-propagation

$$\min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^{N} L(h(x_i; \boldsymbol{\theta}), y_i)$$

Convolutional network

$$\underbrace{\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \lambda \frac{1}{N} \sum_{i=1}^{N} \nabla L(h(x_i; \boldsymbol{\theta}), y_i)}_{\text{Gradient descent update}}$$

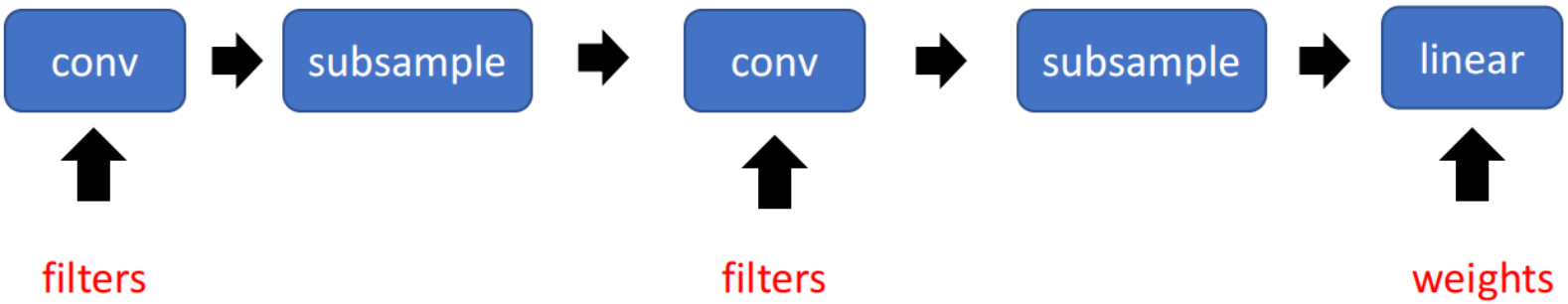
Computing the gradient of the loss

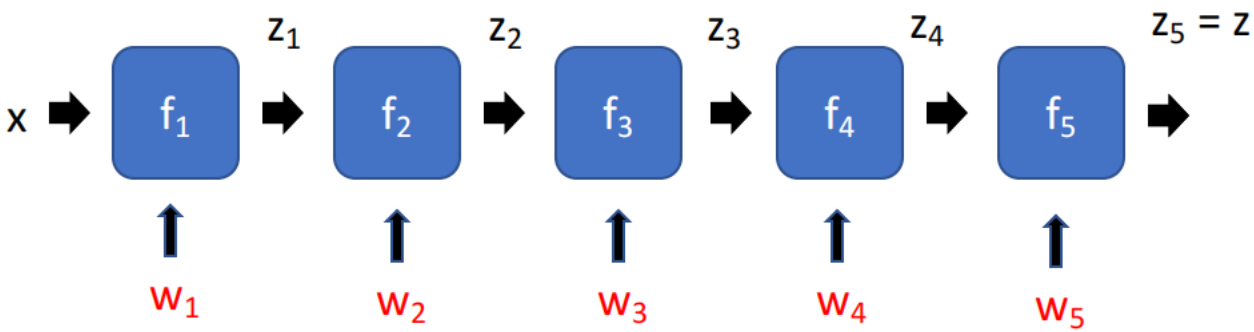
$$\nabla L(h(x;\boldsymbol{\theta}), y)$$

$$z = h(x;\boldsymbol{\theta})$$

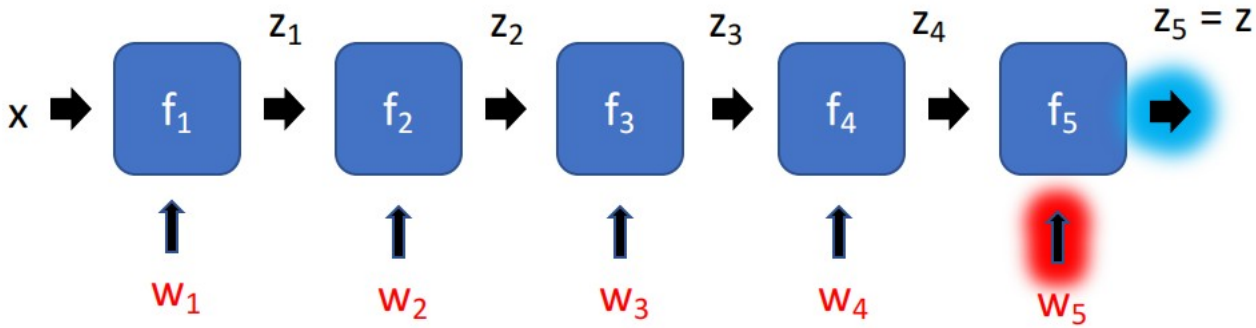
$$\nabla_{\boldsymbol{\theta}} L(z, y) = \frac{\partial L(z, y)}{\partial z} \frac{\partial z}{\partial \boldsymbol{\theta}}$$

Computing the gradient of the loss



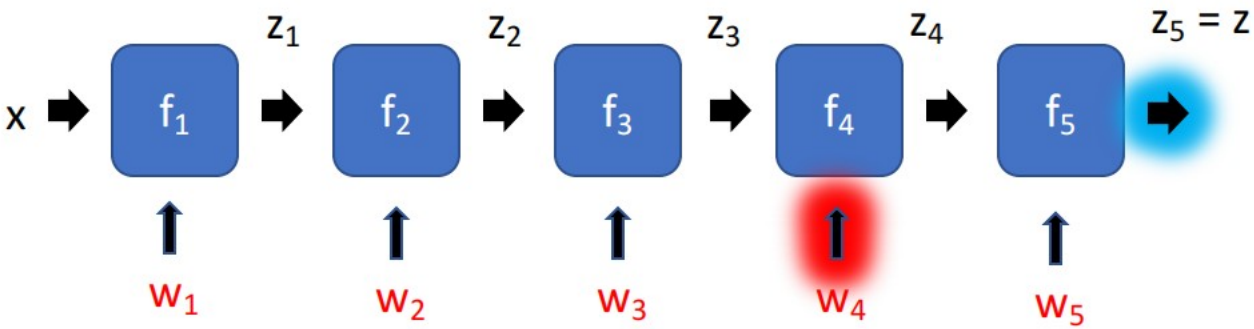


Computing the gradient of the loss



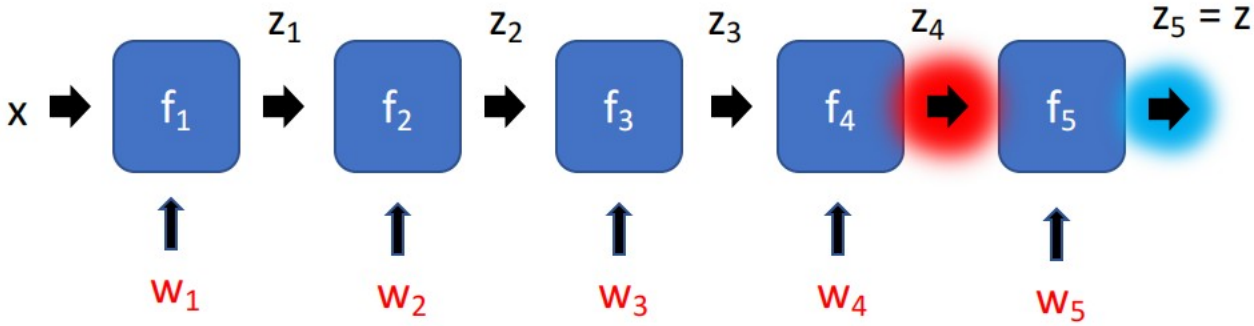
$$\frac{\partial z}{\partial w_5}$$

Computing the gradient of the loss



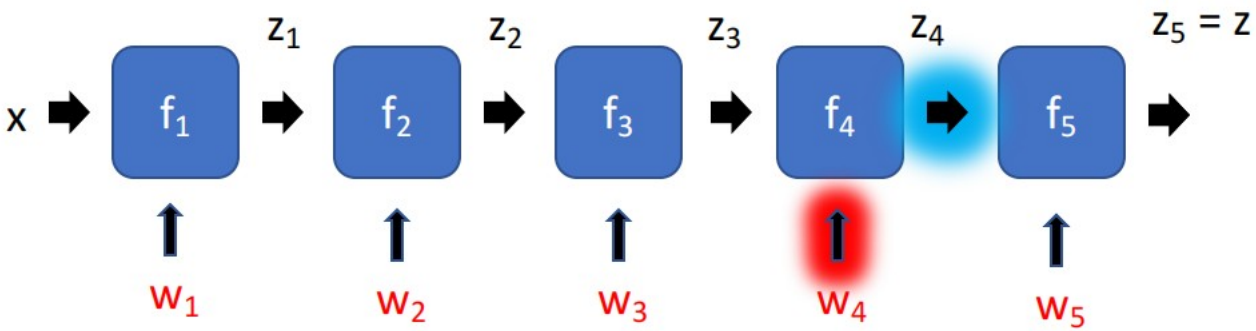
$$\frac{\partial z}{\partial w_4}$$

Computing the gradient of the loss



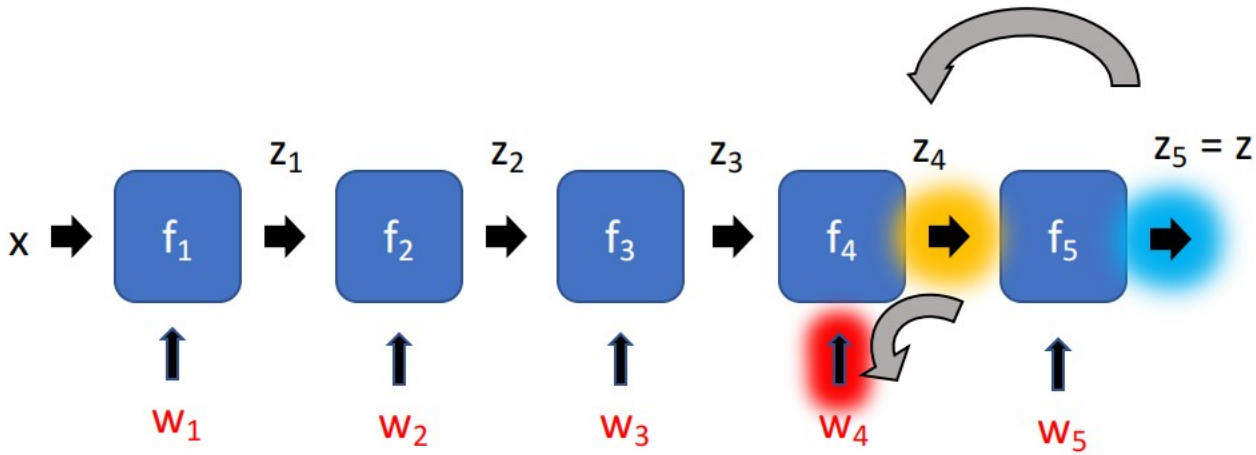
$$\frac{\partial z}{\partial w_4} = \frac{\partial z}{\partial z_4}\frac{\partial z_4}{\partial w_4}$$

Computing the gradient of the loss



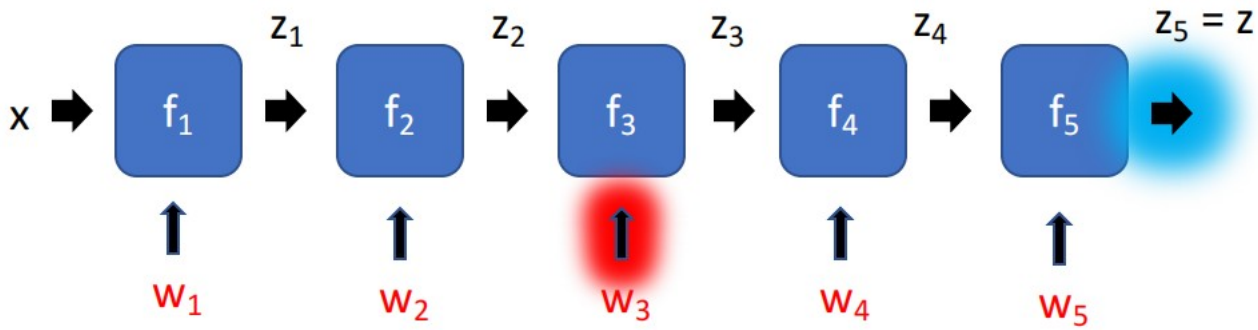
$$\frac{\partial z}{\partial w_4} = \frac{\partial z}{\partial z_4} \frac{\partial z_4}{\partial w_4} \; :$$

Computing the gradient of the loss



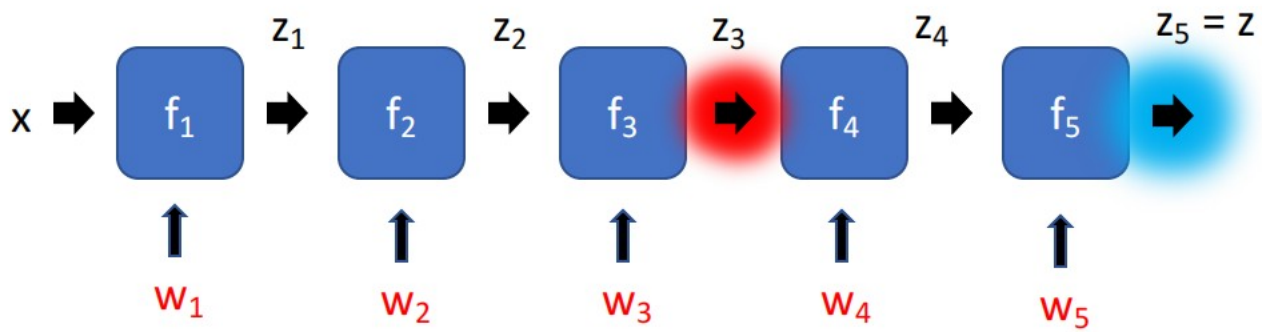
$$\frac{\partial z}{\partial w_4} = \frac{\partial z}{\partial z_4}\frac{\partial z_4}{\partial w_4} = \frac{\partial f_5(z_4, w_5)}{\partial z_4}\frac{\partial f_4(z_3, w_4)}{\partial w_4}$$

Computing the gradient of the loss



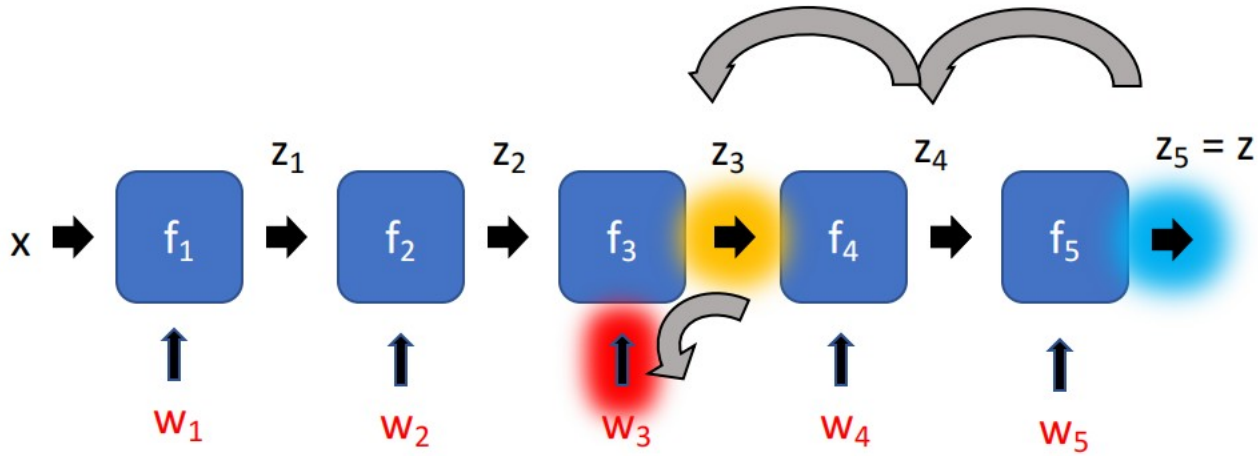
$$\frac{\partial z}{\partial w_3}$$

Computing the gradient of the loss



$$\frac{\partial z}{\partial w_3} = \frac{\partial z}{\partial z_3}\frac{\partial z_3}{\partial w_3}$$

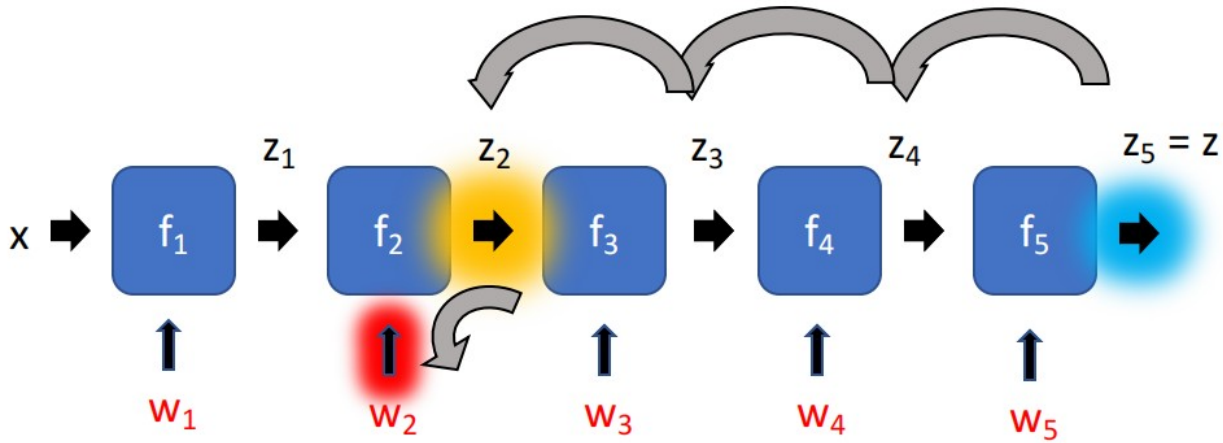
Computing the gradient of the loss



$$\frac{\partial z}{\partial z_3} = \frac{\partial z}{\partial z_4}\frac{\partial z_4}{\partial z_3}$$

$$\frac{\partial z}{\partial w_3} = \frac{\partial z}{\partial z_3}\frac{\partial z_3}{\partial w_3}$$

Computing the gradient of the loss

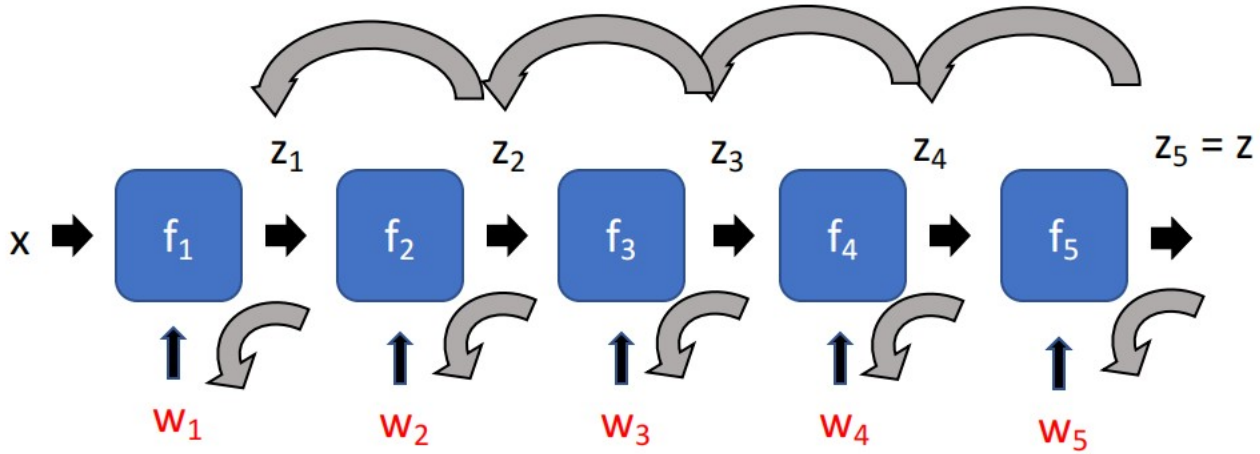


$$\frac{\partial z}{\partial z_2} = \frac{\partial z}{\partial z_3}\frac{\partial z_3}{\partial z_2}$$

$$\frac{\partial z}{\partial w_2} = \frac{\partial z}{\partial z_2}\frac{\partial z_2}{\partial w_2}$$

Recurrence going backward!!

Computing the gradient of the loss



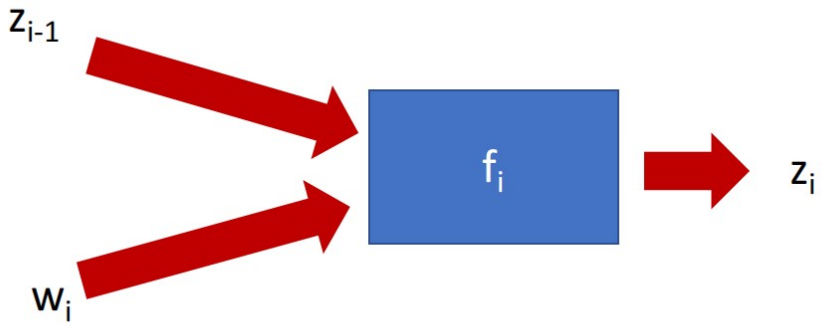
Back-Propagation

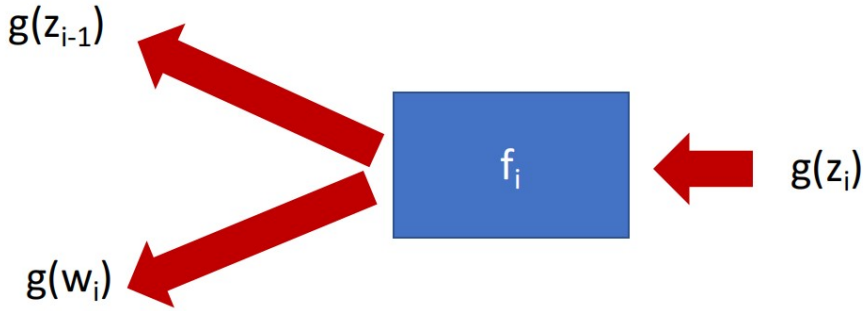
Backpropagation for a sequence of functions

- Each “function” has a “forward” and “backward” module
- Forward module for f_i
  - takes z_i-1 and weight w_i as input
  - produces z_i as output
- Backward module for f_i
  - takes $g(z_i)$ as input
  - produces $g(z_i\text{-}1)$ and $g(w_i)$ as output

$$g(z_{i-1}) = g(z_i)\frac{\partial z_i}{\partial z_{i-1}} \qquad g(w_i) = g(z_i)\frac{\partial z_i}{\partial w_i}$$

Backpropagation for a sequence of functions





Dealing with multiple classes

- If we need to classify inputs into C different classes, we put C units in the last layer to produce C one-vs.-others scores
- Apply softmax function to convert these scores to probabilities:

$$\mathrm{softmax}(f_1, \ldots, f_c) = \left(\frac{\exp(f_1)}{\sum_j \exp(f_j)}, \ldots, \frac{\exp(f_c)}{\sum_j \exp(f_j)} \right)$$

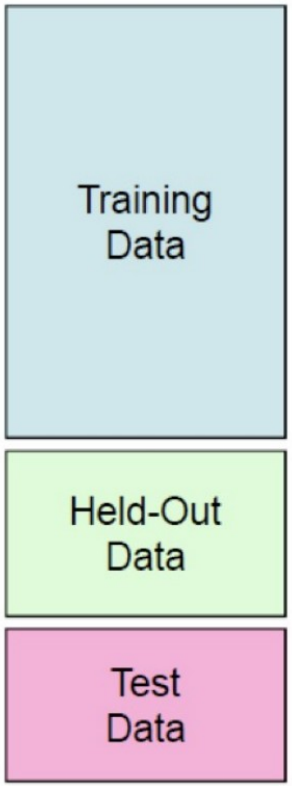
- Use log likelihood (cross-entropy) loss:

$$l(\mathbf{x}_i, y_i; \mathbf{w}) = -\log P_{\mathbf{w}}(y_i \mid \mathbf{x}_i)$$

Best practices for training classifiers

- Goal: obtain a classifier with good generalization or performance on never before seen data

  - 1-Learn parameters on the training set
  - 2-Tune hyperparameters (implementation choices) on the held out validation set
  - 3-Evaluate performance on the test set
  - • Crucial: do not peek at the test set when iterating steps 1 and 2!

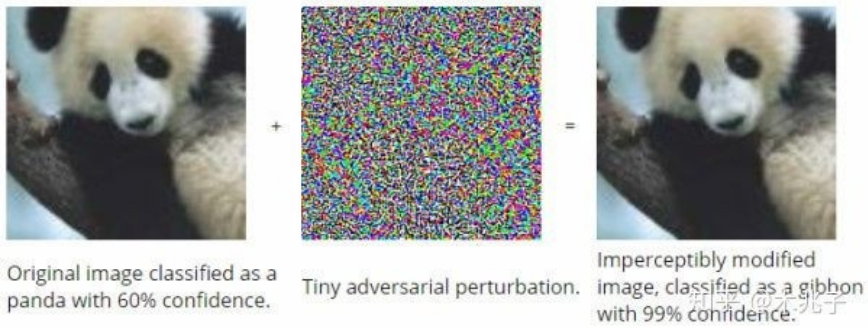


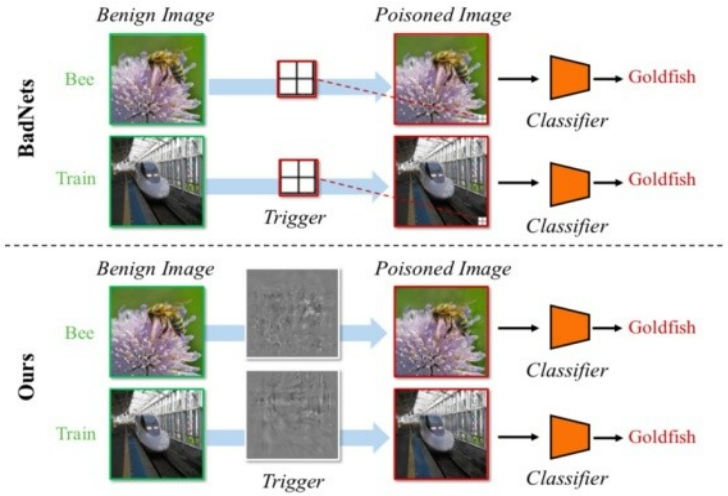
Considerations

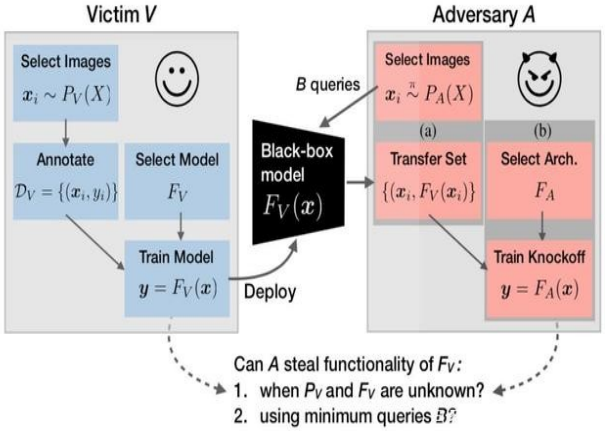
- 1. Data
- 2. Supervision
- 3. Loss functions
- 4. Optimization / Initialization

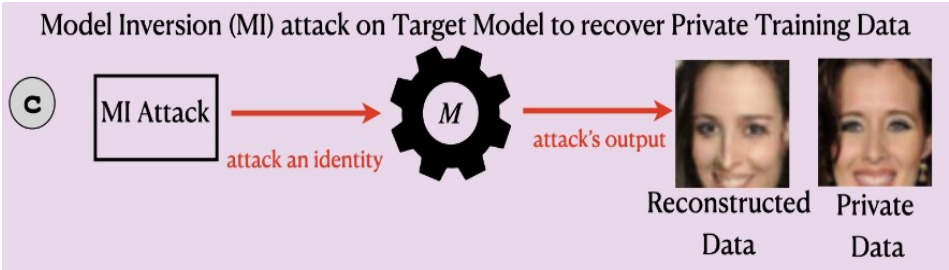
Considerations

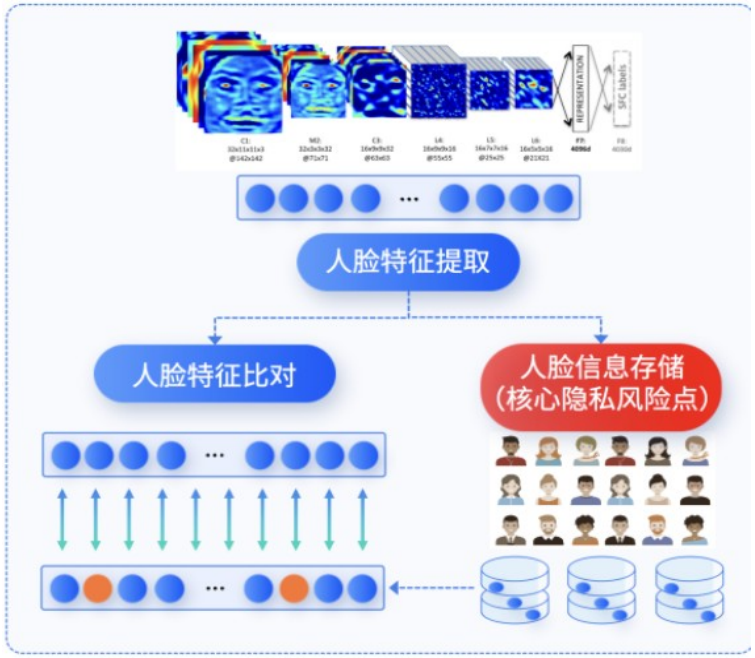
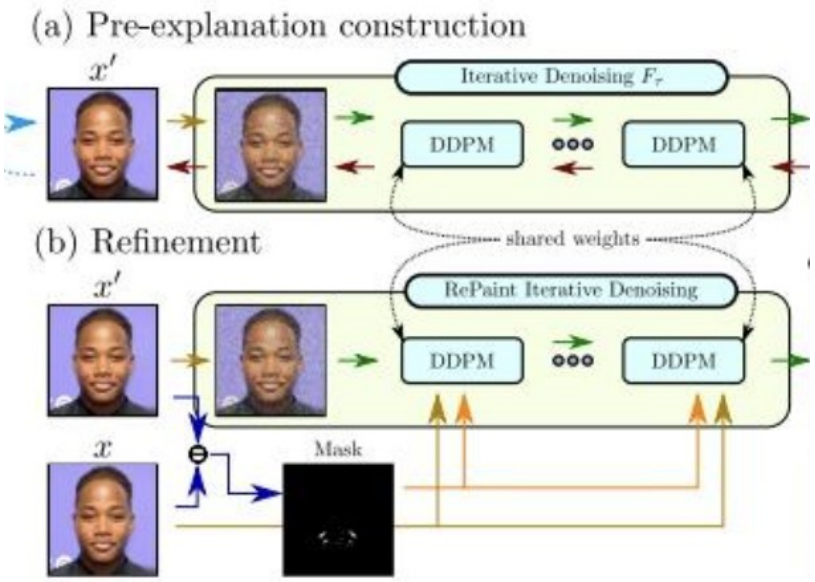
- 1. Data









Considerations

- 1. Data